

Especificação do Projeto Final - Mini-SCADA da Estação de Bombeamento

1. Objetivo do projeto

A dupla deve construir uma aplicação em duas camadas para simular e supervisionar uma **Estação de Bombeamento Inteligente**.

O projeto deve integrar:

- um dispositivo em C++ que simula sensores, bombas, comandos, regras e eventos;
- um supervisor em Python/Streamlit que apresenta leituras, histórico, comandos, alarmes e resumo operacional;
- comunicação por JSON Lines em arquivo ou por TCP;
- histórico em CSV ou SQLite;
- testes, documentação e defesa técnica.

O foco principal da avaliação é Programação Orientada a Objetos: modelagem, encapsulamento, herança, polimorfismo, abstração, composição, coleções, tratamento de exceções, padrões de projeto e organização de código.

2. Escopo e carga de trabalho

O projeto final foi dimensionado para uma dupla trabalhar por aproximadamente duas semanas.

Uma referência mínima de planejamento:

Etapa	Esforço sugerido	Entregável
Planejamento e divisão de tarefas	2h	issues, responsabilidades e arquitetura inicial
Modelo C++ do dispositivo	6h	sensores, bombas, comandos e regras
Supervisor Python/Streamlit	5h	tabela, gráfico, resumo, alarmes e comandos
Persistência e contrato JSON	4h	CSV ou SQLite, README e validação
Testes e tratamento de erros	4h	testes de regra, contrato e integração simples
Variação própria da dupla	3h	cenário, parâmetros e falhas específicas
Revisão, demonstração e defesa	4h	apresentação, AI_LOG e ajustes finais

O total sugerido é de pelo menos **28h** de trabalho combinado da dupla. A divisão não precisa ser exatamente igual em todas as etapas, mas os dois integrantes devem ter commits, issues ou pull requests que mostrem participação técnica real.

3. Componente próprio da dupla

Toda dupla deve criar uma **assinatura operacional**. Essa assinatura define parâmetros e eventos que diferenciam o projeto das demais equipes.

Use um identificador simples da dupla, como:

`ID_DUPLA = soma dos dois últimos dígitos das matrículas dos integrantes`

Se a disciplina não usar matrícula no repositório, use outro identificador combinado definido pela dupla e registrado no README.

Parâmetros derivados da assinatura

O `ID_DUPLA` deve alterar pelo menos quatro elementos do projeto:

Elemento	Como variar
estacao	identificador como EB-07 , EB-13 ou EB-24
nível baixo	limite entre 20% e 35%
nível alto	limite entre 75% e 90%
pressão alta	limite entre 5.0 bar e 8.0 bar
falha simulada	vazamento, sensor travado, bomba bloqueada, pressão elevada ou leitura ruidosa
regra extra	alternância de bombas, bloqueio por motor, alerta por vazão baixa ou modo manutenção

Exemplo de registro no README:

Item	Valor da dupla
Identificador da estação	EB-17
Nível baixo	27%
Nível alto	84%
Pressão alta	6.8 bar
Falha obrigatória	sensor de nível travado por 10 ciclos
Regra extra	bloquear partida quando motor estiver em alerta

Esses valores devem aparecer no código, nos testes, no supervisor e na defesa.

4. Requisitos obrigatórios

O projeto final deve possuir:

- pelo menos 4 tipos de sensores;
- pelo menos 2 bombas ou 1 bomba com 1 atuador auxiliar claramente modelado;
- pelo menos 4 regras de controle, incluindo 1 regra específica da dupla;
- pelo menos 4 comandos de atuação;
- pelo menos 3 tipos de alarme;

- pelo menos 1 falha simulada específica da dupla;
- contrato JSON documentado;
- uso explícito de herança;
- uso explícito de polimorfismo;
- uso explícito de encapsulamento;
- uso de pelo menos 2 padrões de projeto;
- supervisor em Streamlit;
- histórico em CSV ou SQLite;
- testes cobrindo regras principais e parâmetros da dupla;
- `README.md` atualizado;
- `AI_LOG.md` preenchido;
- histórico de issues, commits ou pull requests mostrando divisão de trabalho;
- apresentação técnica final.

5. Critérios arquiteturais

Conceito	Evidência esperada no projeto
Classes e objetos	sensores, bombas, estação, comandos e alarmes instanciados
Encapsulamento	atributos protegidos e métodos de acesso controlados
Herança	<code>Sensor</code> como base de sensores especializados
Polimorfismo	coleção de sensores ou comandos tratados por interface comum
Abstração	interfaces para estratégia, comando, repositório ou observador
Composição	<code>EstacaoBombeamento</code> coordenando objetos internos
Tratamento de exceções	leitura inválida, arquivo ausente ou comando bloqueado
Coleções	sensores, comandos, leituras e alarmes em listas ou vetores
Organização	pastas separadas por responsabilidade

6. Padrões de projeto

A equipe deve usar pelo menos dois padrões de projeto. A escolha deve ser justificada pela necessidade do projeto.

Técnica/Padrão	Melhor uso	Esforço	Entregável	Limitação
Factory	criação de sensores ou componentes por tipo/configuração	médio	ponto único de criação documentado	pode ser excesso com poucos tipos fixos
Strategy	regras de controle intercambiáveis	médio	regras separadas do controlador principal	exige configuração clara
Observer	eventos e alarmes sem acoplamento direto	médio	notificadores/observadores identificáveis	precisa evitar notificações duplicadas
Command	atuações registradas, enfileiradas ou testadas	médio	comandos de atuação como objetos ou estrutura equivalente	aumenta número de classes
State	estados com transições relevantes	médio/alto	estados e transições documentados	exige cuidado para não complicar
Repository	persistência isolada do restante do sistema	médio	camada clara de acesso a CSV ou SQLite	exige disciplina de uso

Na apresentação, a equipe deve mostrar onde os padrões aparecem no código e responder:

- que problema concreto o padrão resolveu;
 - que alternativa mais simples foi considerada;
 - por que a solução escolhida não é apenas complexidade adicional;
 - quais classes ou módulos participam do padrão;
 - como a implementação foi testada.
-

7. Contrato JSON

O projeto final pode manter o contrato inicial por leitura ou evoluir para um pacote com múltiplas leituras, comandos e alarmes. Em qualquer caso, o `README.md` deve documentar:

- campos obrigatórios;
- campos opcionais;
- tipos de dados;
- unidades;
- exemplos de valores válidos;
- exemplos de status;
- significado de comandos e alarmes;
- frequência esperada de envio ou gravação;
- parâmetros derivados da assinatura operacional da dupla.

Contrato mínimo por leitura

Campo	Obrigatório	Tipo esperado	Observação
<code>tag</code>	sim	texto	identificador do sensor ou ativo
<code>valor</code>	sim	número	leitura simulada
<code>unidade</code>	sim	texto	unidade de grandeza
<code>timestamp</code>	sim	texto	instante real da leitura, gerado no momento da aquisição
<code>status</code>	sim	texto	estado operacional padronizado

A equipe pode adicionar campos como `tipo`, `estacao`, `bomba`, `comandos`, `alarmes` e `qualidade`, desde que o contrato esteja documentado e o supervisor trate esses campos de forma consistente.

8. Supervisor Streamlit

O supervisor deve apresentar, no mínimo:

- tabela de leituras atuais;
- gráfico histórico de pelo menos duas variáveis;

- resumo do estado das bombas ou atuadores;
- lista de alarmes;
- lista ou histórico de comandos.

Não há layout obrigatório. A avaliação considera se a interface ajuda a entender o estado da estação, não se ela copia um modelo específico.

Durante a demonstração, a equipe deve conseguir mostrar:

- atualização das leituras;
- histórico consultável;
- mudança de estado das bombas ou atuadores;
- geração de pelo menos um comando;
- exibição de pelo menos um alarme;
- comportamento diante de leitura inválida ou situação de falha simulada.

9. Divisão de trabalho da dupla

A entrega deve mostrar que os dois integrantes participaram tecnicamente.

Integrante	Responsabilidade principal	Evidência mínima
Integrante A	dispositivo C++, regras, comandos e testes C++	commits ou PRs em <code>dispositivo_cpp/</code>
Integrante B	supervisor, persistência, validação JSON e testes Python	commits ou PRs em <code>supervisor_python/</code>
Ambos	contrato JSON, arquitetura, README, AI_LOG e apresentação	revisões cruzadas e decisões registradas

A divisão pode ser diferente, mas deve estar registrada no README. Cada integrante deve conseguir explicar uma parte que escreveu e uma parte que revisou.

10. Resistência a cópia e uso indevido de IA

O objetivo não é impedir consulta a materiais ou ferramentas, mas garantir autoria, entendimento e capacidade de defesa.

Requisitos de rastreabilidade

- O README deve conter a assinatura operacional da dupla.
- O `AI_LOG.md` deve registrar prompts, respostas aproveitadas, trechos rejeitados e justificativa técnica.
- As issues ou pull requests devem mostrar pequenas entregas incrementais.
- Os testes devem usar os parâmetros específicos da dupla.
- A apresentação deve incluir uma alteração curta feita ao vivo ou explicada em detalhe, escolhida a partir do código entregue.

Itens que aumentam suspeita de cópia

- parâmetros iguais aos de outra dupla sem justificativa;
- README genérico que não menciona a assinatura operacional;
- código com padrões de projeto citados, mas sem uso real;
- testes que não verificam as regras próprias da dupla;
- integrante que não consegue explicar código presente no repositório;
- `AI_LOG.md` vazio ou incompatível com o histórico de commits.

11. README e AI_LOG

README mínimo

O `README.md` deve conter:

- descrição do projeto;
- arquitetura em duas camadas;
- diagrama de classes;
- contrato JSON;
- instruções para compilar C++;
- instruções para executar Streamlit;
- instruções para testes;
- assinatura operacional da dupla;
- divisão de responsabilidades;
- decisões de padrões de projeto;
- limitações conhecidas.

AI_LOG mínimo

O `AI_LOG.md` deve registrar:

Campo	O que escrever
Data	quando a IA foi usada
Ferramenta	agente, chat, IDE ou outra ferramenta
Pedido	o que foi solicitado
Aceito	o que entrou no projeto
Rejeitado	o que foi descartado
Justificativa	por que a equipe aceitou ou rejeitou

Uso de IA sem entendimento não será aceito. A equipe deve defender tecnicamente o que entregou.

12. Critério de avaliação

A nota do trabalho será composta por duas partes:

Parte da avaliação	Peso	Tipo de nota	Evidência
Implementação conforme especificação	35%	compartilhada pela dupla	repositório, execução, contrato JSON, supervisor, testes, README, AI_LOG e histórico de trabalho
Entrevista presencial do trabalho	65%	individual	explicação técnica, defesa de decisões, leitura de código, testes, alterações solicitadas e entendimento da própria entrega

A nota da implementação será a mesma para os dois integrantes da dupla. A nota da entrevista será individual.

Cada dupla deve agendar junto ao professor um horário para apresentar o trabalho presencialmente. Na entrevista, cada integrante deve demonstrar domínio do projeto, incluindo partes que implementou diretamente e partes que revisou.

Critérios da implementação

Critério	Evidência
Modelagem POO	classes, relações, encapsulamento, herança e polimorfismo
Funcionamento do sistema	simulação, supervisor, histórico, comandos e alarmes
Complexidade da dupla	assinatura operacional, falha própria e regra extra
Padrões de projeto	pelo menos dois padrões aplicados com justificativa
Testes e validação	testes de regra, contrato, parâmetros da dupla e integração simples
Documentação e rastreabilidade	README, contrato JSON, AI_LOG, issues e commits

Critérios da entrevista individual

Critério	Evidência
Entendimento do domínio	explica sensores, bombas, regras, comandos e alarmes
Leitura do código	localiza classes, métodos, fluxo de dados e pontos de extensão
Defesa de arquitetura	justifica padrões, relações entre classes e separação C++/Python
Testes e validação	explica testes existentes e interpreta falhas
Autoria e rastreabilidade	relaciona commits, issues, AI_LOG e decisões aceitas ou rejeitadas
Adaptação supervisionada	responde a uma pequena mudança ou pergunta técnica sobre o projeto

13. Roteiro da apresentação técnica

Cada equipe deve apresentar:

1. problema e escopo da estação;
2. arquitetura geral;
3. diagrama de classes;
4. contrato JSON;

5. assinatura operacional da dupla;
6. demonstração do dispositivo C++;
7. demonstração do supervisor Streamlit;
8. histórico, comandos e alarmes;
9. falha simulada e regra específica da dupla;
10. testes;
11. uso de IA e rastreabilidade;
12. principais decisões e limitações.

Tempo sugerido: 10 a 12 minutos por dupla, mais a entrevista individual de cada integrante.

14. Checklist de entrega

- ☐ Pelo menos 4 tipos de sensores.
 - ☐ Pelo menos 2 bombas ou 1 bomba com 1 atuador auxiliar.
 - ☐ Pelo menos 4 regras de controle.
 - ☐ Pelo menos 4 comandos de atuação.
 - ☐ Pelo menos 3 tipos de alarme.
 - ☐ Falha simulada específica da dupla.
 - ☐ Assinatura operacional documentada.
 - ☐ Contrato JSON documentado.
 - ☐ Herança, polimorfismo e encapsulamento evidentes.
 - ☐ Pelo menos 2 padrões de projeto.
 - ☐ Supervisor Streamlit funcional.
 - ☐ Histórico em CSV ou SQLite.
 - ☐ README atualizado.
 - ☐ AI_LOG preenchido.
 - ☐ Testes executáveis.
 - ☐ Issues, commits ou pull requests mostram participação dos dois integrantes.
 - ☐ Apresentação técnica preparada.
-

15. Perguntas de defesa

1. Onde a equipe usou polimorfismo de forma concreta?
2. Quais padrões de projeto foram usados e que problema cada um resolveu?
3. O que o supervisor sabe sobre o dispositivo e o que ele não deveria saber?
4. Qual decisão de arquitetura poderia ser simplificada em uma versão menor do projeto?
5. Que teste demonstra que uma regra de controle importante não foi quebrada?
6. Quais parâmetros pertencem especificamente a esta dupla?
7. Que parte do código precisaria mudar se o limite de pressão da dupla fosse alterado?
8. Como a falha simulada aparece no dispositivo, no histórico e no supervisor?